

Algoritmo di Peterson

Metodo software per risolvere il problema della mutua esclusione, garantendo che solo un processo alla volta acceda alla sezione critica.

- Ogni processo usa un **flag** per segnalare la volontà di entrare.
- Una variabile **turn** stabilisce chi può entrare per primo.
- Se un processo trova che l'altro è già nella sezione critica, attende il proprio turno.

Algoritmo di Dekker

Prima soluzione software alla mutua esclusione, senza istruzioni hardware speciali.

- Due variabili **flag** indicano l'intenzione di entrare.
- Una variabile **turn** alterna l'accesso.
- Un processo entra solo se:
 - La sua flag = vera
 - L'altra flag = falsa
 - Oppure è il suo turno.

Algoritmo del Banchiere

Previene il deadlock verificando se una richiesta può essere soddisfatta senza rischi.

Dati richiesti

- **R** = numero totale di risorse
- **V** = risorse disponibili
- **C** = massimo richiesto da ogni processo
- **A** = risorse già assegnate

Fasi

1. Inizializza: $W = V$.
2. Controlla se esiste un processo P_i eseguibile ($Richiesta \leq W$). Se sì:
 - Segna P_i come completato.
 - Rilascia le risorse in W .
 - Ripeti.
3. Se tutti i processi terminano → **stato sicuro**.
Altrimenti → **stato insicuro**.

Algoritmo safe()

- Simula l'esecuzione di processi con richieste compatibili.
- Se `rest` è vuoto alla fine → stato sicuro.

Buddy System

Divide la memoria in blocchi di potenze di 2.

- All'inizio tutta la memoria è un blocco unico.
- Se un processo richiede memoria:
 - Trova il blocco più piccolo $\geq$ richiesta.
 - Se troppo grande, divide in due "buddy".
- Quando due buddy liberi sono adiacenti, si fondono.

Riduce frammentazione esterna causando, però frammentazione interna.

Best-Fit

Sceglie il blocco libero più piccolo sufficiente alla richiesta. Minimizzando frammentazione interna.

First-Fit

Assegna il **primo** blocco libero abbastanza grande.

Next-Fit

Simile al First-Fit, ma la ricerca parte dall'ultima allocazione.

FIFO (First In, First Out)

Rimuove la pagina più vecchia. Semplice, ma può eliminare pagine ancora utili.

LRU (Least Recently Used)

Rimuove la pagina meno usata di recente. Segue il principio di località, ma è difficile da implementare.

Optimal (OPT)

Rimuove la pagina che non servirà per più tempo nel futuro. Minimizza i page fault, ma è impossibile da implementare realmente.

Clock

Variante di FIFO con **bit di utilizzo**.

- Se il bit = 1 → lo azzerà e passa avanti.
- Se il bit = 0 → rimuove la pagina.

First Come First Served (FCFS)

- **Politica:** Non-preemptive.
- **Vantaggi:** semplice, overhead minimo.
- **Svantaggi:** effetto convoglio con processi lunghi.

Round Robin (RR)

- **Politica:** Preemptive con **quantum**: Un intervallo di tempo fisso che il sistema operativo assegna a ciascun processo.
- **Vantaggi:** equo, ottimo per sistemi multiutente.
- **Svantaggi:** overhead alto se quantum troppo piccolo.

Shortest Job Next (SJN/SPN)

- **Politica:** Non-preemptive.
- **Vantaggi:** minimizza tempo medio di completamento.
- **Svantaggi:** rischio starvation.

Shortest Remaining Time (SRT)

- **Politica:** Preemptive, esegue il processo col minor tempo rimanente.
- **Vantaggi:** ottimizza tempi medi.
- **Svantaggi:** forte rischio di starvation.

HRRN (Highest Response Ratio Next)

- **Politica:** Non-preemptive.
- **Formula:**
- **Vantaggi:** bilancia efficienza e equità.
- **Svantaggi:** più complesso da implementare.

Priority Scheduling

- **Politica:** processi con priorità maggiore vengono serviti prima.
- **Svantaggi:** rischio starvation.

Multilevel Queue

- **Politica:** suddivide i processi in più code con algoritmi diversi.
- **Esempio:** processi di sistema, interattivi, batch.

SSTF (Shortest Seek Time First)

Serve la richiesta più vicina alla testina. Riduce tempi medi di ricerca, ma con rischio starvation per richieste lontane.

SCAN (Ascensore)

La testina si muove in una direzione servendo richieste, poi inverte. Bilanciando i tempi di attesa.

C-SCAN (Circular SCAN)

- **Funzione:** La testina scorre in una sola direzione; ritorna all'inizio senza servire richieste.
- **Vantaggi:** attese più uniformi rispetto a SCAN.

N-Step-SCAN

- **Funzione:** Divide le richieste in sotto-code, servendone una per volta.
- **Vantaggi:** evita lunghe attese.

FSCAN

- **Funzione:** Usa due code: una attiva e una in attesa.
- **Vantaggi:** stabilizza la coda, riduce ritardi.